Table of Contents

1 How did we get here?

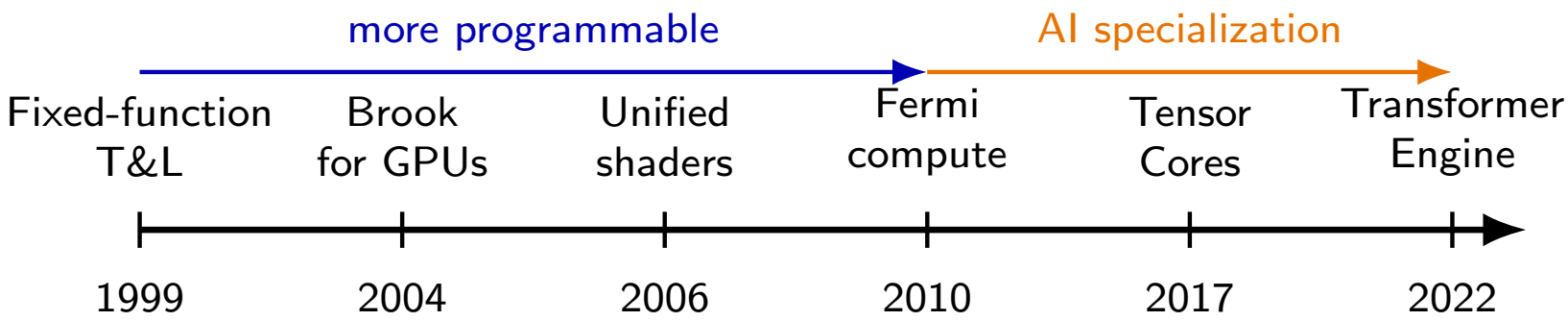
Timeline

- **1999—2003:** fixed-function graphics gives way to programmable vertex and pixel shaders.
- **2004:** Brook for GPUs demonstrates a high-level stream-programming model on top of graphics APIs.
- **2006—2007:** GeForce 8800 introduces a unified shader architecture; NVIDIA releases CUDA publicly in 2007.
- **2010:** Fermi adds a coherent cache hierarchy, ECC support, and much stronger support for C++ and scientific computing.
- **2017:** Volta introduces Tensor Cores for mixed-precision matrix multiply–accumulate.
- **2020—2024:** Ampere broadens Tensor Core formats; Hopper adds the FP8 Transformer Engine; Blackwell adds further low-precision and AI-specific hardware.

What shaped modern GPU architecture?

Driver of architectural change

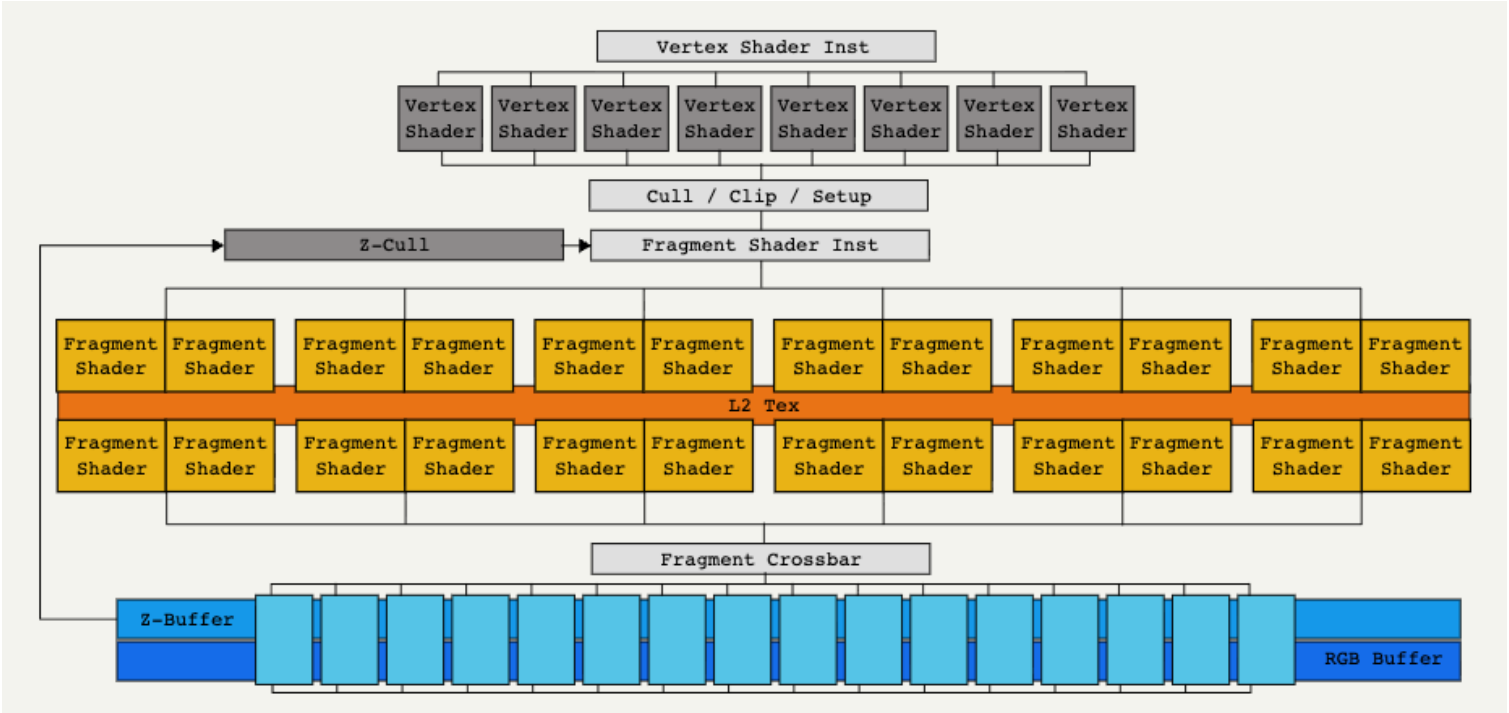
New workloads cause generalization/programmability, established workloads cause specialization.



What shaped modern GPU architecture?

$<$2006: graphics first

- GeForce 256 (1999) accelerated fixed-function transform and lighting.
- Later chips added programmable vertex and pixel shaders, but kept distinct stages and resources.



The G71 architecture (2005). Diagram source: Sanglard (2020).

What shaped modern GPU architecture?

2004: GPGPU before CUDA. Demand was always there.

- Brook for GPUs compiled a stream language to graphics APIs.
- It showed useful non-graphics workloads could exploit GPU throughput, despite an awkward programming model.

Brook for GPUs: Stream Computing on Graphics Hardware

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan
Stanford University

Abstract

In this paper, we present Brook for GPUs, a system for general-purpose computation on programmable graphics hardware. Brook extends C to include simple data-parallel constructs, enabling the use of the GPU as a streaming co-processor. We present a compiler and runtime system that abstracts and virtualizes many aspects of graphics hardware. In addition, we present an analysis of the effectiveness of the GPU as a compute engine compared to the CPU, to determine when the GPU can outperform the CPU for a particular algorithm. We evaluate our system with five applications, the SAXPY and SGEMV BLAS operators, image segmentation, FFT, and ray tracing. For these applications, we demonstrate that our Brook implementations perform comparably to hand-written GPU code and up to seven times faster than their CPU counterparts.

CR Categories: I.3.1 [Computer Graphics]: Hardware Architecture—Graphics processors D.3.2 [Programming Languages]: Language Classifications—Parallel Languages

Keywords: Programmable Graphics Hardware, Data Parallel Computing, Stream Computing, GPU Computing, Brook

modern hardware. In addition, the user is forced to express their algorithm in terms of graphics primitives, such as textures and triangles. As a result, general-purpose GPU computing is limited to only the most advanced graphics developers.

This paper presents *Brook*, a programming environment that provides developers with a view of the GPU as a streaming coprocessor. The main contributions of this paper are:

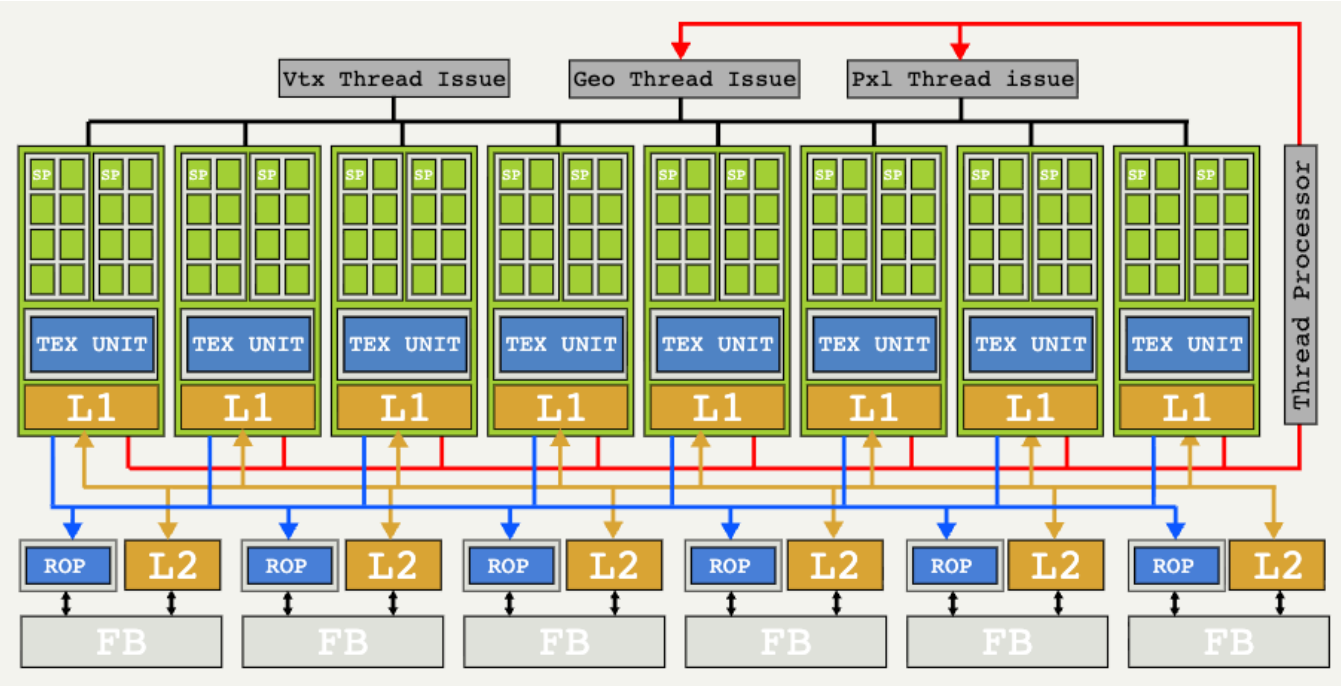
- The presentation of the Brook stream programming model for general-purpose GPU computing. Through the use of streams, kernels and reduction operators, Brook abstracts the GPU as a streaming processor.
- The demonstration of how various GPU hardware limitations can be virtualized or extended using our compiler and runtime system; specifically, the GPU memory system, the number of supported shader outputs, and support for user-defined data structures.
- The presentation of a cost model for comparing GPU vs. CPU performance tradeoffs to better understand under what circumstances the GPU outperforms the CPU.

Buck et al., "Brook for GPUs," *ACM TOG* 23(3), 2004.

What shaped modern GPU architecture?

2006-2007: increasing complexity necessitates unified compute

- G80 ran vertex, geometry, and pixel work on unified pool of **Streaming Multiprocessors** (SM's).
- **This is the heart of modern GPU architecture.**



G80 block diagram. Diagram source: Sanglard (2020); architecture described by Lindholm et al. (2008).

Our model of the GPU (Execution)

The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

Our model of the GPU (Execution)

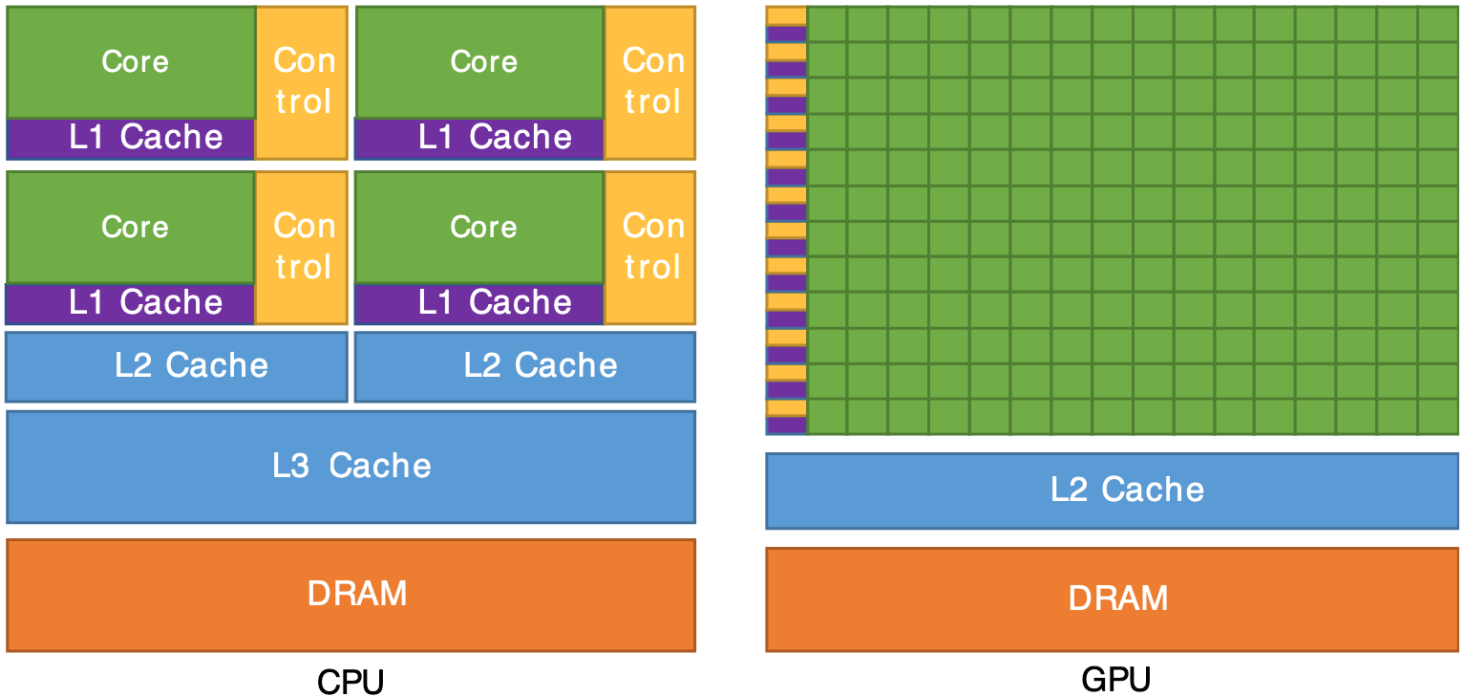
The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

The CUDA Cores execute the same instruction in parallel on different data.

Our model of the GPU (Execution)

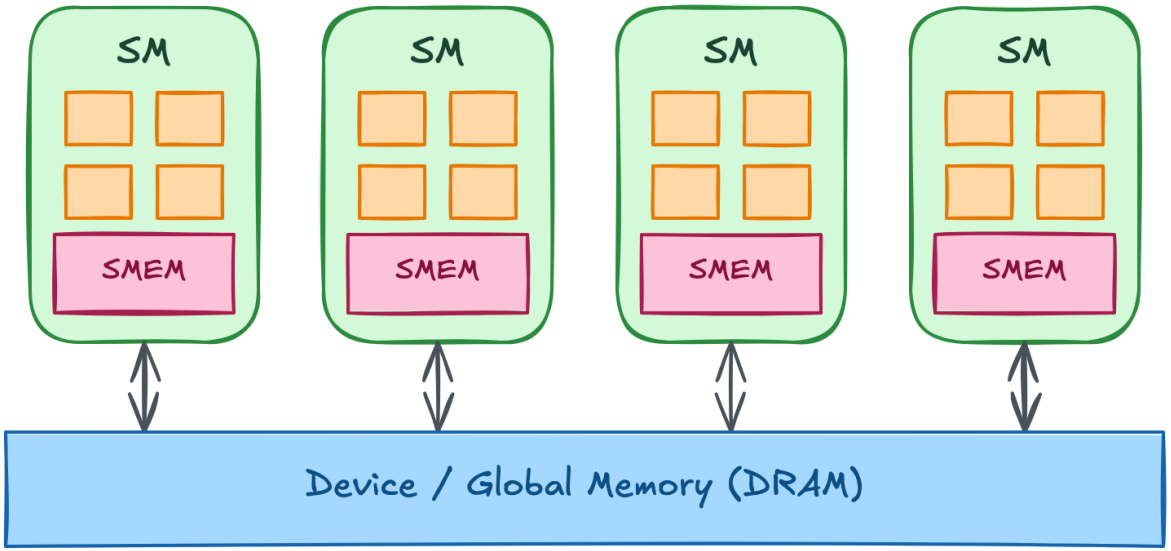
The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

The CUDA Cores execute the same instruction in parallel on different data.



Our model of the GPU (Memory)

The GPU has its own memory called **Device Memory (DRAM)**. Each SM has a **programmable L1 Cache** which we also call **Shared Memory (SMEM)**.

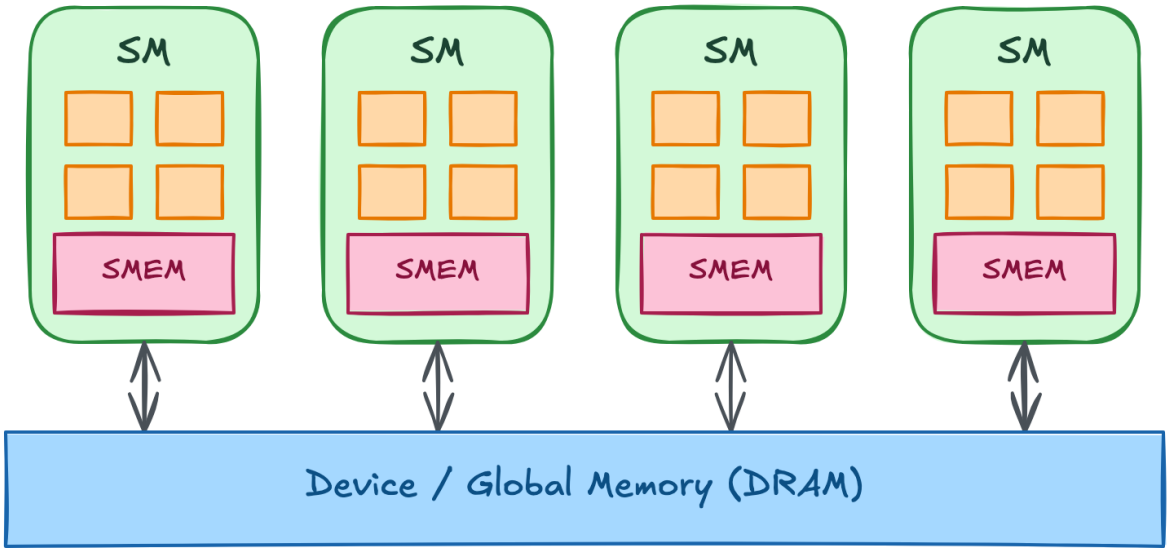


Performance

Accessing to DRAM is orders of **tens of times** slower than accessing SMEM. Effective use of SMEM is crucial to performance.

Takeaways

- SMs: primary compute unit
- CUDA Cores: like the "ALUs" of the SMs.
- L1 cache/Shared memory: cache for the SM
- Device/Global Memory: like RAM for a normal CPU system.



Whats next

How can we actually implement a matrix multiplication on a GPU? What tools and mental models do we need?